

MOBILE IMPLEMENTATION PLAN

ViewTree on NVIDIA Jetson

Purpose: Detailed engineering plan for the mobile runtime of the paper's "Mobile Execution" / "Mobile Runtime Design" sections (\$overview-runtime, \$runtime) on NVIDIA Jetson — the deferred M6 phase of the roadmap. It maps every mechanism in the paper text (scene reuse, pose-indexed caches, shared KV prefixes, sibling batching, phase scheduling, the (B_t, D_t, C_t) scheduler, unsupported-view and exhausted-budget fallbacks) onto the stack that is already implemented and trained in this repository (viewtree/, scripts/depth/), so that **the trained models run unchanged** and only the execution substrate is new. **Status:** Plan (nothing in this document is built yet). Numbers marked *est.* are pre-profiling estimates to be replaced by W0-W1 measurements; all others come from the server-side implementation record (DESIGN_IMPLEMENTATION.md, RESULTS.md). **Date:** 31 August 2026. **Design rule (from the paper):** the VLM confidence decides *which* camera paths are useful; the scheduler decides *how many* useful paths the device can afford. The same trained model must operate from B_t = 1 (single-path, low-power) to B_t = 3 (full beam) without any change to its reasoning interface.

Executive summary

The runtime is four subsystems on top of the existing reasoning code: (1) a **scene store** that amortizes VGGT reconstruction across questions and versions it by coverage; (2) a **two-level cache** — pose-indexed renders + encoded vision tokens, and a paged, read-only **shared KV prefix** per branch point; (3) a **batched execution engine** (TensorRT-LLM for Qwen2.5-VL-7B W4A16, TensorRT FP16 for the ViT and VGGT, a CUDA point-splat renderer) that groups sibling renders/encodes per tree level and separates memory-heavy phases; (4) a **profile-driven scheduler** mapping deadline, battery mode, free memory, and thermal state to (B_t, D_t, C_t) = (beam width, remaining depth, cost allowance), with degradation that only reduces *future* exploration. Primary target: **Jetson AGX Orin 64 GB** (full system incl. 32-frame reconstruction); constrained profile: **Orin NX 16 GB** (16-frame reconstruction, B_t ≤ 2, weights hot-swapped). Six workpackages (W0-W5, ≈ 6–8 engineer-weeks) end with paired accuracy parity vs the server implementation and Pareto curves of accuracy vs measured latency/energy across nvpmodel power states — the evaluation the original design's H5/H6 hypotheses require.

1. Target platforms and software stack

	Jetson AGX Orin 64 GB (primary)	Jetson Orin NX 16 GB (constrained)
GPU	2048-core Ampere, 64 Tensor Cores	1024-core Ampere, 32 Tensor Cores
Memory	64 GB LPDDR5, 204.8 GB/s (unified)	16 GB LPDDR5, 102.4 GB/s (unified)
Power states	MAXN / 50 W / 30 W / 15 W (nvpmodel)	25 W / 15 W / 10 W
Role	Full system: 32-frame reconstruction, B_t ≤ 3, D_t ≤ 3	Reduced: 16-frame reconstruction, B_t ≤ 2, D_t ≤ 2, phase-exclusive residency

Software: JetPack 6.x (L4T r36, Ubuntu 22.04, CUDA 12.x), TensorRT 10, **TensorRT-LLM (Jetson build)** for the VLM, PyTorch aarch64 wheel for VGGT and the renderer during bring-up (replaced by TensorRT engines in W1-W2), ONNX Runtime for the heads. Telemetry: tegrastats (thermal zones, rail power via INA3221), /proc/meminfo, NVML where exposed; profiling with Nsight Systems for the phase-overlap decisions in §5. Fallback VLM runtime if TensorRT-LLM's Qwen2.5-VL multimodal path proves immature on Jetson: llama.cpp (GGUF Q4_K_M) with its Qwen2-VL vision path, or MLC-LLM — the cache/scheduler design below is runtime-agnostic (see risk R1).

2. Model deployment plan (per module)

Module	Server form	Jetson form	Memory (est.)
VLM backbone	Qwen2.5-VL-7B + LoRA r=16 (bf16)	LoRA merged into base per adapter , then W4A16 AWQ via TensorRT-LLM; one engine per shipped adapter (best depth-1 D_10k, ViewTree-D SFT-C); paged KV cache in FP8	≈ 5.5 GB weights + KV (\$4)
Vision encoder (ViT)	inside Qwen2.5-VL	Separate TensorRT FP16 engine with profiles for the two input classes: context frames and 448 ² -capped renders; emits the vision tokens that both the cache (\$4.1) and the VLM consume	≈ 1.3 GB
Confidence / value head	MLP 3584→512→1 (GELU, dropout), temperature-calibrated	ONNX → TensorRT FP16 (or CPU; < 2 M params, μs-scale). Temperature is refit on-device against W4A16 outputs on the calibration split — quantization shifts logits and the head reads hidden states (risk R4)	negligible
Scene builder	VGGT-1B bf16, 32 frames (≈ 7.8 GB + 0.22 GB/frame activations)	FP16 TensorRT (fallback: torch). AGX: 32 frames (≈ 15 GB peak, fits). NX: 16 frames (≈ 11 GB peak) run phase-exclusive (VLM weights not resident; \$5)	2.5 GB weights + activations
Renderer	torch z-buffer point-splat, splat 2, few ms	Port to a single CUDA kernel (scatter-min z-buffer + splat); render at the controller's 448 ² input cap, not full frame — cuts ViT cost ~4×	< 0.5 GB
Pose proposer / action mask	viewtree/render.py human_poses, viewtree/posebank.py (walkable hull, 12 spots × 8 yaws + top-down, coverage ≥ 45 %)	Reused verbatim (NumPy/CPU); poses computed once per scene version and stored with the scene (\$3)	negligible

The reasoning code itself — gate → branch → keep-2 → fuse → arbitrate (depth-1) and the beam walk of `scripts/depth/run_tree_d.py` — is refactored into a runtime-neutral **tree driver** with an execution-backend interface (`render(pose_batch)`, `encode(image_batch)`, `prefill(prefix_id, suffix_tokens)`, `score_actions(state)`, `head(hidden)`), implemented once for the server (current torch path, used for parity tests) and once for Jetson.

3. Scene store: amortized, versioned reconstruction

Implements "Reusing the Reconstructed Scene".

1. **Session model.** A *scene session* owns: the fused point cloud + colors (unified memory), per-frame camera poses/intrinsics, the walkable hull + validity mask, the pose bank for the current version, and a monotonically increasing `scene_version`.
2. **Reconstruction trigger.** Reconstruction runs at session start and afterwards only when new capture leaves current coverage: the trigger is the fraction of recent frames whose estimated pose lies outside the stored walkable hull (or whose view frustum overlaps stored points below a threshold, est. < 40 %) over a sliding window. Nearby questions between triggers reuse the store — a user asks several questions about a room and pays reconstruction once (server-side precedent: one reconstruction serves all of a scene's questions).
3. **Incremental update.** On trigger, VGGT runs on the frame buffer (32 / 16 frames by platform); the new fragment is aligned to the store by the shared frames' poses (Umeyama on camera centers, est. sufficient for static rooms), fused, then `scene_version += 1` — which deliberately invalidates the pose-indexed caches (\$4.1) and the pose bank, both rebuilt lazily.
4. **Memory pressure.** The store targets a point budget (AGX est. 40 M points ≈ 1.4 GB in xyz+rgb fp16/u8; NX 10 M). Above budget: voxel-downsample far regions first (distance

from current walkable hull), then evict regions not touched by any cached question of the session. Under critical pressure (scheduler signal, §6) the store drops to a "compact" representation: downsampled cloud + the 97 pose-bank JPEGs, which alone can still serve the discrete action space.

5. **Visibility handling.** The renderer returns image + coverage mask; pixels with no supporting points are painted to a reserved background value and the mask is attached to the cache entry. A view with coverage < 45 % is **unsupported** (§7) — the mask threshold is the same constant the models were trained with; it must not be retuned on device.

4. Cache organization

4.1 Pose-indexed observation cache (render + tokens)

Key = (scene_version, pose_key) where pose_key is the *discrete pose-bank index* whenever the pose came from the bank (the ViewTree-D action space is exactly the 97-entry bank, so cache keys are small integers), else the quantized tuple (walkable-grid cell, yaw bin/8, pitch bin, fov, render resolution). Two value classes, separately evictable:

- **L1 render cache:** JPEG (448², q≈85, ≈ 60 KB) + coverage mask + validity bit. Full 97-view bank ≈ 6 MB/scene — pin it.
- **L2 token cache:** post-ViT vision tokens for the render (fp16, est. ≈ 0.4–0.8 MB/view depending on token count). Budget-bounded LRU (AGX est. 256 entries ≈ 200 MB; NX 64). A hit skips both the renderer *and* the ViT — the dominant per-view cost.

Hits are expected from: beam paths converging on the same bank pose (observed in server traces), consensus re-answers, repeated questions in a session, and the top-down view (shared by every question of a scene). Context-frame tokens are cached the same way keyed by frame id — they are reused by *every* call of *every* question in the session.

4.2 Shared KV prefix across branches

Implements "Sharing and Batching Work across Paths" with TensorRT-LLM's paged KV cache + block-reuse:

1. Per question, the runtime prefills once the **root prefix**: system prompt + 8 context-frame tokens + question. Its KV pages are marked read-only; the gate call and the direct answer reuse them.
2. At a branch point, each child path owns only its **suffix pages** (its render tokens + pose-tag text + its answer/action tokens). Page table: copy-on-write references to the shared prefix, so memory grows O(prefix + Σ suffixes) instead of O(B·prefix).
3. When the head prunes a path, its suffix pages are freed immediately (the paper's requirement); the prefix persists until the question completes.
4. KV budget check (AGX, fp8 KV, 28 layers · 512 kv-dim est. ≈ 28 KB/token): prefix ≈ 3–4k tokens ≈ 110 MB shared; each live suffix ≈ 0.5–1k tokens ≈ 25 MB; B_t = 3, depth 3 = worst-case ≈ 350 MB — comfortably inside budget; NX halves it via B_t ≤ 2 and 16-frame prefixes.
5. If the runtime cannot expose page sharing (fallback runtimes), emulate by **prefix replay batching**: keep the prefix tokens, re-prefill them batched with all sibling suffixes in one call, and count the measured overhead in C_t (the design-doc's stated contingency).

4.3 Action scoring without decoding

The controller's action choice (STOP/moves) is read from the **prefill logits of the last position** (exactly how `run_tree_d.py:action_scores` works) — no autoregressive decode. Answers decode ≤ 12 tokens. Decoding is therefore a minor cost on Jetson; prefill and ViT dominate, which is why §4.1/

§4.2 are the two mechanisms that matter (server evidence: mean 4.5 calls/question, up to ~20 at full depth).

5. Batching and phase scheduling

1. **Per-level sibling batching.** The tree driver already advances level-by-level; at each level it emits all sibling poses at once → one renderer launch (CUDA kernel batched over views), one ViT batch (profile for batch 1-6), one batched VLM prefill sharing the root prefix. This turns the 3-6 small calls per level into 3 large launches.
2. **Phase separation.** Four phases with different bottlenecks: reconstruction (compute+memory heavy, rare), rendering (tiny), visual encoding (compute), VLM prefill/decode (memory-bandwidth). Rules: reconstruction never overlaps VLM execution (on NX the VLM engine is not resident during reconstruction; on AGX it stays resident but idle); render+encode of level $k+1$ candidates may overlap the decode of level k 's answers **only when** the scheduler's contention check passes (free bandwidth headroom and GPU util < threshold from the W4 profile); otherwise strictly sequential — matching the paper's "separate memory-heavy stages under contention".
3. **CPU/GPU overlap.** Path management (beam bookkeeping, head calls if on CPU, cache lookups, pose masking) runs on CPU threads overlapped with accelerator execution; it is μ s-ms scale and never blocks the GPU queue.
4. **Thermal-aware execution.** A telemetry thread samples `tegrastats` at 2 Hz; sustained-load tests (W5) decide whether overlap must additionally be disabled above a temperature threshold per power mode.

6. Scheduler: $(B_t, D_t, C_t) = S(\pi_t)$

1. **Offline profile (per device × nvpmode state × engine).** Measured once (W4): reconstruction latency/energy per frame count; render per view; ViT per batch size; VLM prefill per 1k tokens (with/without prefix reuse); decode per token; peak memory of each phase. Stored as a lookup table shipped with the app.
2. **Runtime inputs π_t :** application deadline, battery mode (maps to nvpmode state), free memory (`/proc/meminfo` + allocator watermark), thermal state (zone temps + throttle flags).
3. **Admission at question start:** compute the *predicted* cost of the cheapest useful plan (gate + direct answer) and of one full level; set D_t = largest depth whose worst-case predicted cost fits the deadline/energy budget; B_t = min(3, paths affordable at that depth, memory-limited path count from §4.2); C_t = remaining allowance in profiled cost units. Low-power mode pins $B_t = 1$ (greedy walk — the trained model runs single-path without interface change); an empty budget pins $D_t = 0$ (direct answer only; the gate still runs because it is the direct path).
4. **Per-step ledger.** After every phase, C_t is decremented by *measured* cost (EMA-corrected predictions). If a step overruns, the scheduler reduces only the future: first D_t (shallower), then B_t (narrower), then denies further acquisitions entirely. **Acquired views are never dropped** — they stay in the paths and caches (paper requirement).
5. **Exhausted capacity.** When B_t or C_t reaches its floor mid-question, the VLM is constrained to {continue one retained path, FUSE-equivalent (answer from retained evidence), STOP}: implemented by masking the action set handed to `score_actions` — the same masking machinery the models were trained with, so no distribution shift.
6. **Division of labor (invariant):** the head ranks paths (keep-k unchanged at min(k, B_t)); the scheduler only sets B_t, D_t, C_t . No scheduler signal ever enters the head's score.

7. Unsupported views and fallbacks

1. A rendered candidate with coverage < 45 % is marked **invalid**: it is never encoded, never enters the beam (server behavior preserved), and the action that produced it is removed from the valid set at that state so the controller re-chooses a smaller movement or another direction (FORWARD → TURN_* typically). The VLM never sees unsupported regions as observations (mask-fill before encode, §3.5).
2. If all paths hit their limits, the runtime returns the answer of the highest-value retained state **with its calibrated confidence** in the API response. The application layer may then: prompt the user to capture another view (which will trigger a scene update, §3.2), physically move, or escalate to a cloud endpoint. All three stay outside the core runtime; the API just exposes (answer, confidence, trace, budget_report).

8. Memory budgets

Component	AGX Orin 64 GB	Orin NX 16 GB
VLM engine (W4A16) + ViT (FP16)	≈ 7 GB resident	≈ 7 GB, evicted during reconstruction
KV cache pool	1 GB pool (worst case ≈ 0.35 GB)	0.5 GB
VGGT weights + peak activations	2.5 + ≈ 13 GB (32 frames, transient)	2.5 + ≈ 8.5 GB (16 frames, transient, phase-exclusive)
Scene store + pose bank + caches	≈ 2 GB	≈ 0.6 GB
Headroom (OS, camera, app)	> 35 GB	≈ 2–3 GB

NX is feasible only with phase-exclusive residency (VLM engine load ≈ 2–4 s *est.* amortized over a session's single reconstruction) — this is the main reason reconstruction frequency (§3.2) matters.

9. Workpackages, exit criteria, measurement protocol

WP	Work	Exit criterion
W0 bring-up (1 wk)	JetPack 6, torch pipeline runs unmodified (bf16→fp16), 10-question smoke test	End-to-end answers on AGX; baseline per-phase latency table
W1 engines (1–2 wk)	Merge LoRA, AWQ W4A16 TRT-LLM engine; ViT TRT engine; head ONNX + on-device temperature refit	Parity: ≥ 99 % answer agreement and head-AUROC within 0.01 vs server on 200 held-out questions
W2 scene store (1 wk)	VGGT TRT/torch on device, versioned store, update trigger, renderer CUDA kernel	32-frame (AGX) / 16-frame (NX) reconstruction within memory budget; render < 5 ms/view
W3 caches + batching (1 wk)	Pose/token caches, KV prefix sharing (or replay fallback), per-level batching	Measured: 2nd question in a scene ≥ 2× faster than 1st (<i>est.</i>); B=3 level ≤ 1.6× cost of B=1 (<i>est.</i>)
W4 scheduler (1 wk)	Offline profiler across nvpmmodel states; (B_t,D_t,C_t) controller; ledger + degradation	Deadline compliance ≥ 95 % across injected budget shocks; no acquired-view drops
W5 evaluation (1–2 wk)	Full study on VSI held-out subset (paired vs server), 3 power modes × {AGX, NX}	Accuracy within CI of server; Pareto curves accuracy vs p50/p95 latency and energy/question; 30-min sustained-load thermal run

Protocol (from the original design §8.8/§8.10): warm and cold measured separately; reconstruction reported both amortized and upfront; p50/p95/p99 latency, energy per question (INA3221 rails), peak memory, cache hit rates, steady-state temperature; per-phase breakdown (reconstruction / render / encode / prefill / decode / management); equal-answer-quality operating points compared, not just max throughput; budgets tuned on validation scenes only.

10. Risks and mitigations

#	Risk	Mitigation
R1	TensorRT-LLM Qwen2.5-VL multimodal support immature on Jetson	Runtime-agnostic backend interface (§2); fallbacks llama.cpp GGUF / MLC-LLM; prefix-replay batching replaces KV page sharing (§4.2.10) with measured overhead
R2	VGGT activation memory on NX	16-frame cap; chunked attention if needed; phase-exclusive residency; AGX remains the reference platform
R3	Unified-memory contention between phases	Phase separation rules (§5.12) are the default-off position; overlap is enabled only where Nsight shows headroom
R4	W4A16 shifts hidden states → head miscalibration	W1 refits temperature (and, if AUROC drops > 0.02, retrains the 2-layer head) on device-generated states; parity gate blocks W2+ until passed
R5	Thermal throttling breaks deadline predictions	Profile per nvpmode state incl. throttled regime; scheduler reads throttle flags and switches to the throttled cost table
R6	Cache staleness after scene updates	scene_version in every key; invalidation is total by construction, rebuild is lazy
R7	Pareto gain too small vs naive sequential execution	Pre-registered reframe (original design §10.1): report as active-view reasoning system with on-device feasibility, not a systems-contribution paper

Repository anchors

Tree drivers to refactor: depth-1 tree ([viewtree/tree.py](#)), beam walk ([scripts/depth/run_tree_d.py](#)); pose machinery reused as-is: [viewtree/render.py](#) (human_poses, coverage mask), [viewtree/posebank.py](#) (bank, transitions, action mask); models: [checkpoints/scale/D_highcost_10k](#), [checkpoints/depth/sft_c](#), [checkpoints/conf_head_v2_human.pt](#), [checkpoints/depth/value_head.pt](#). Companion documents: [DESIGN_IMPLEMENTATION.md](#) §7 (built-vs-deferred table this plan completes), [ViewTree_Research_Design_Document.pdf](#) §7 (original mobile design).